Sekans Modellerine Sezgisel Bir Yolculuk: RNN, LSTM ve GRU Neden Ortaya Çıktı?

Atıl Samancıoğlu

1 Bu Dökümanın Amacı

Bu döküman, Transformer mimarisine geçmeden önce **neden RNN ile başladığımızı**, **neden LSTM ve GRU'ya ihtiyaç duyduğumuzu** sezgisel ve mantıksal bir akışla anlatmak için hazırlandı.

Buradaki hedefimiz:

- RNN, LSTM ve GRU'yu mimari olarak ezberlemek değil
- Her birinin **hangi problemi çözmek için** tasarlandığını netleştirmek

Matematiğe gireceğiz, ama matematik burada bir **amaç değil**, bir **araç**. Her formülün şu soruya cevap vermesi gerekiyor:

> "Bu formül hangi sezgisel problemi çözüyor?"

2 Tek Bir Ana Örnek: Cümleyi Anlamak

Tüm döküman boyunca tek bir örnek üzerinden ilerleyeceğiz:

> *"I did not say he stole the money."*

Bu cümle, sekans modellerinin neden zorlandığını anlamak için ideal çünkü:

- Anlam, tek tek kelimelerde değil
- Kelimeler arasındaki **zamansal ve anlamsal ilişkide**

saklı.

Bu cümleyi kelime kelime işleyen bir modelin, baştaki "not" bilgisini sona kadar taşıması gerekiyor.

3 Recurrent Neural Networks (RNN)

3.1 RNN'nin Temel Fikri

RNN'nin çıkış noktası çok doğaldır:

> "Bir diziyi okurken, geçmişte okuduklarımı tamamen unutmamalıyım."

Bu fikir matematiksel olarak şöyle ifade edilir:

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$$

Bu denklemde:

- x_t: t anındaki kelime (örneğin "not")
- h_{t-1}: geçmişin özeti
- h_t: geçmiş + şu an

3.2 Bu Tasarım Neden Mantıklıydı?

Dil, zamanla akan bir yapıdır. Bir kelimeyi anlamak için genellikle önceki kelimeleri bilmek gerekir.

RNN şunu yapmaya çalışır:

> "Geçmişi tek bir vektörde özetle ve bu özeti sürekli güncelle."

Bu, kısa cümleler için gayet iyi çalışır.

3.3 Ama Nerede Kırıldı?

Sorun, cümle uzadıkça ortaya çıkar.

Ana örneğimize bakalım:

> *I <u>did not</u> say he stole the money*

"not" kelimesi, cümlenin anlamını tamamen değiştiriyor. Ama "not" bilgisi, "money" kelimesine gelene kadar defalarca tanh ve matris çarpımlarından geçiyor.

Sonuç:

- Bilgi zayıflıyor
- Model, uzun vadeli bağımlılıkları unutuyor

Bu, **sequence problemi**nin ilk net hali.

4 Vanishing Gradient Problemi (Kısa Ama Net)

Backpropagation sırasında gradyanlar, zaman boyunca geriye doğru akar.

Çok sayıda tanh veya sigmoid kullanıldığında:

$$\frac{\partial h_t}{\partial h_{t-k}} \to 0$$

Bu şu anlama gelir:

> "Model, geçmişteki hatalardan öğrenemez hale geliyor."

Bu problem RNN'nin yapısal bir sınırıydı.

5 LSTM: "Hatırlamayı Öğrenelim"

5.1 LSTM'nin Ortaya Çıkış Sebebi

LSTM şu soruyla doğdu:

> "Her bilgiyi taşımak zorunda mıyım? Hangisini tutacağımı öğrenemez miyim?"

Bu soru, RNN'den çok daha güçlü bir fikirdir.

5.2 LSTM Hücresinin Sezgisi

LSTM, tek bir hidden state yerine iki kavram kullanır:

- Cell state (c_t): Uzun vadeli hafıza
- Hidden state (h_t): Anlık çıktı

Cell state, neredeyse bir "bilgi otoyolu" gibi davranır.

5.3 Kapılar (Gates) Neyi Çözüyor?

LSTM üç temel kapı kullanır:

- Forget gate: "Bunu unutmalı mıyım?"
- Input gate: "Yeni bilgiyi eklemeli miyim?"
- Output gate: "Dışarı neyi göstermeliyim?"

Örneğin forget gate:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

Bu formül şunu yapar:

> "Geçmiş bilginin hangi kısmı önemli, hangisi değil?"

5.4 Ana Örneğe Geri Dönelim

"not" kelimesi geldiğinde:

- Forget gate, önceki varsayımları silebilir
- Input gate, olumsuzluk bilgisini cell state'e güçlü şekilde yazabilir

Bu sayede "not" bilgisi, cümlenin sonuna kadar daha sağlam taşınır.

5.5 Ama Temel Sorun Hâlâ Var

LSTM çok güçlüdür ama şunu değiştirmez:

> "Hâlâ sırayla işliyoruz."

Yani:

- Paralel işlem yok
- Uzun sekanslarda maliyet yüksek

6 GRU: "Bu Kadar Kapıya Gerek Var mı?"

6.1 GRU'nun Felsefesi

GRU şu soruyla ortaya çıktı:

> "LSTM iyi ama fazla karmaşık. Daha sade bir şey yapabilir miyiz?"

GRU, LSTM'deki bazı kapıları birleştirir.

6.2 GRU'nun Temel Mekanizması

GRU'da iki ana kapı vardır:

- Update gate
- Reset gate

Update gate, şu soruya cevap verir:

> "Geçmiş mi daha önemli, yeni bilgi mi?"

Bu sadeleşme sayesinde:

- Daha az parametre
- Daha hızlı eğitim

elde edilir.

6.3 Sezgisel Karşılaştırma

- RNN: Her şeyi taşımaya çalışır
- LSTM: Ne taşınacağını öğrenir
- GRU: Aynı fikri daha sade uygular

Ama üçünün de ortak noktası şudur:

7 Kritik Duvar: Sıralı İşleme

Tüm bu modellerde şu varsayım vardır:

> "Bir kelimeyi anlamak için önceki kelimeleri sırayla işlemeliyim."

Bu varsayım, bizi şu noktaya getirir:

- Uzun sekanslarda yavaşlık
- Bilginin dolaylı taşınması

Ve işte tam burada şu soru doğar:

8 Sonraki Adım

> "Bir kelimeyi anlamak için neden tüm kelimelere aynı anda bakmayalım?"

Bu soru bizi doğrudan Attention fikrine götürür.

Bir sonraki dökümanda, kaldığımız yerden devam edeceğiz ve **Attention'ın bu sıralı zorunluluğu nasıl kırdığını** adım adım inşa edeceğiz.